XML Cheatsheet — Държавен изпит ФМИ

---

1. Добре-структуриран XML (Well-formed XML)

Задължителни правила:

- Един **коренен елемент** (root element)
- Всеки отварящ таг `<tag>` има затварящ `</tag>` или е self-closing `<tag/>`
- Елементите **не се припокриват**: `<a><b></b></a>` ✅ | `<a><b></a></b>` ❌
- Имената са **case-sensitive**: `<book>` ≠ `<Book>`
- Атрибутите имат стойности в кавички: `id="123"` ✅ | `id=123` ❌
- Дублиращи се атрибути в един елемент — **забранено**
- Имената започват с буква или `_`, не с `xml`
- Запазени символи: `&` = `&`, `<` = `<`, `>` = `>`, `"` = `"`, `'` = `'`

```xml
<?xml version="1.0" encoding="UTF-8"?>
<root>
  <element attribute="value">text content</element>
  <empty-element/>
</root>
```

XML Namespace (пространство от имена):

```xml
<book xmlns="http://example.com/books"
      xmlns:auth="http://example.com/authors">
  <auth:name>Ivan Ivanov</auth:name>
</book>
```

---

2. DTD (Document Type Definition)

Кога се пита: Даден е XML документ → напиши DTD. Или: даден е DTD → напиши валиден XML.

Структура на DTD

```
<?xml version="1.0"?>
<!DOCTYPE root-element [
  <!ELEMENT ...>
  <!ATTLIST ...>
  <!ENTITY ...>
  <!NOTATION ...>
]>
```

Деклариране на елементи

Синтаксис	Значение
`<!ELEMENT name (#PCDATA)>`	Само текст

`<!ELEMENT name EMPTY>`	Без съдържание (self-closing)
`<!ELEMENT name ANY>`	Произволно съдържание
`<!ELEMENT name (child1, child2)>`	Точно тези деца в този ред
`<!ELEMENT name (child1 | child2)>`	Едно от двете
`<!ELEMENT name (#PCDATA | child)*>`	Смесено съдържание

Индикатори за брой:

Символ	Значение
(нищо)	Точно 1 път
`?`	0 или 1 пъти
`*`	0 или повече пъти
`+`	1 или повече пъти

Деклариране на атрибути

```
<!ATTLIST element-name
  attr-name  attr-type  default-value
>
```

Типове атрибути:

Тип	Значение
`CDATA`	Текстова стойност
`ID`	Уникален идентификатор
`IDREF`	Реферира ID на друг елемент
`IDREFS`	Списък от IDREF (разделени с интервал)
`NMTOKEN`	XML токен
`ENTITY`	Реферира ENTITY
`(val1|val2)`	Изброени стойности

Стойности по подразбиране:

Синтаксис	Значение
`#REQUIRED`	Задължителен атрибут
`#IMPLIED`	Незадължителен (optional)
`#FIXED "value"`	Фиксирана стойност

`"default"`	Стойност по подразбиране

Entities и Notation

```
<!ENTITY name "replacement text">          <!-- текстова единица -->
<!ENTITY logo SYSTEM "logo.jpg" NDATA jpeg>  <!-- бинарна (непарсирана) -->
<!NOTATION jpeg PUBLIC "image/jpeg">         <!-- формат на данни -->
```

Пример: Пълно DTD (изпит 2020 — HotelCatalog)

Задача: DTD с дърво: HotelCatalog → {HotelChain*, Hotel+, Service*}, всеки с под-елементи.

```
<?xml version="1.0"?>
<!DOCTYPE HotelCatalog [
  <!ELEMENT HotelCatalog (HotelChain*, Hotel+, Service*)>
  <!ATTLIST HotelCatalog Version CDATA #REQUIRED>

  <!ELEMENT HotelChain (ChainName, ChainOwner?)>
  <!ELEMENT ChainName (#PCDATA)>
  <!ELEMENT ChainOwner (#PCDATA)>
  <!ATTLIST HotelChain id ID #REQUIRED>

  <!ELEMENT Hotel (HotelName, HotelAddress, HotelImg)>
  <!ELEMENT HotelName (#PCDATA)>
  <!ELEMENT HotelAddress (#PCDATA)>
  <!ELEMENT HotelImg EMPTY>
  <!ATTLIST HotelImg src ENTITY #REQUIRED>
  <!ATTLIST Hotel chainRef IDREF #IMPLIED>
  <!ATTLIST Hotel serviceRefs IDREFS #IMPLIED>

  <!ELEMENT Service (ServiceName, ServiceDescr?)>
  <!ELEMENT ServiceName (#PCDATA)>
  <!ELEMENT ServiceDescr (#PCDATA)>
  <!ATTLIST Service id ID #REQUIRED>
  <!ATTLIST Service Included (YES|NO) #IMPLIED>

  <!NOTATION jpeg PUBLIC "image/jpeg">
  <!ENTITY hotelLogo SYSTEM "hotel.jpg" NDATA jpeg>
]>
```

Пример: DTD за рецепти (изпит 2010)

Задача: Даден XML с `<collection>`, `<recipe>`, `<ingredient>`, `<preparation>`, `<step>` → напиши DTD.

```
<!DOCTYPE collection [
  <!ELEMENT collection (description?, recipe+)>
  <!ELEMENT description (#PCDATA)>
  <!ELEMENT recipe (title, ingredient+, preparation, comment?)>
```

```
  <!ELEMENT title (#PCDATA)>
  <!ELEMENT ingredient EMPTY>
  <!ATTLIST ingredient
    name   CDATA #REQUIRED
    amount CDATA #IMPLIED
    unit   CDATA #IMPLIED>
  <!ELEMENT preparation (step+)>
  <!ELEMENT step (#PCDATA)>
  <!ELEMENT comment (#PCDATA)>
  <!ELEMENT nutrition EMPTY>
  <!ATTLIST nutrition
    calories      CDATA #IMPLIED
    fat           CDATA #IMPLIED
    carbohydrates CDATA #IMPLIED
    protein       CDATA #IMPLIED>
]>
```

---

3. XML Schema (XSD)

Предимства пред DTD: по-богат, XML синтаксис, строги типове данни, фасети.

Основна структура

```
<?xml version="1.0" encoding="UTF-8"?>
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="root" type="RootType"/>

  <xs:complexType name="RootType">
    <xs:sequence>
      <xs:element name="child" type="xs:string" minOccurs="1"
maxOccurs="unbounded"/>
    </xs:sequence>
    <xs:attribute name="id" type="xs:ID" use="required"/>
  </xs:complexType>

</xs:schema>
```

Ключови елементи

Елемент	Употреба
`<xs:element>`	Декларира елемент
`<xs:attribute>`	Декларира атрибут
`<xs:complexType>`	Елемент с деца или атрибути
`<xs:simpleType>`	Елемент само с текст + ограничения
`<xs:sequence>`	Деца в точен ред

`<xs:all>`	Деца в произволен ред (всички)
`<xs:choice>`	Точно едно от децата
`<xs:group>`	Именована група елементи
`<xs:attributeGroup>`	Именована група атрибути

Брой срещания

Атрибут	Стойност	DTD еквивалент
`minOccurs="1" maxOccurs="1"`	точно 1	*(нищо)*
`minOccurs="0" maxOccurs="1"`	0 или 1	`?`
`minOccurs="0" maxOccurs="unbounded"`	0+	`*`
`minOccurs="1" maxOccurs="unbounded"`	1+	`+`

Прости типове данни

`xs:string`, `xs:integer`, `xs:decimal`, `xs:boolean`, `xs:date`, `xs:float`, `xs:ID`, `xs:IDREF`, `xs:IDREFS`, `xs:anyURI`

Фасети (ограничения)

```
<xs:simpleType name="AgeType">
  <xs:restriction base="xs:integer">
    <xs:minInclusive value="0"/>
    <xs:maxInclusive value="120"/>
  </xs:restriction>
</xs:simpleType>

<xs:simpleType name="ZipCode">
  <xs:restriction base="xs:string">
    <xs:pattern value="[0-9]{4}"/>
  </xs:restriction>
</xs:simpleType>

<xs:simpleType name="StatusType">
  <xs:restriction base="xs:string">
    <xs:enumeration value="active"/>
    <xs:enumeration value="inactive"/>
  </xs:restriction>
</xs:simpleType>
```

Фасет	Значение
`minInclusive` / `maxInclusive`	≥ / ≤
`minExclusive` / `maxExclusive`	> / <

`length`	точна дължина
`minLength` / `maxLength`	мин/макс дължина
`pattern`	регулярен израз
`enumeration`	изброени стойности

Пример: XML Schema за книга (изпит 2010)

Задача: Даден XML с `<book>`, `<character>` → напиши XML Schema.

```
<xs:schema xmlns:xs="http://www.w3.org/2001/XMLSchema">

  <xs:element name="book">
    <xs:complexType>
      <xs:sequence>
        <xs:element name="title" type="xs:string"/>
        <xs:element name="author" type="xs:string"/>
        <xs:element name="character" type="CharacterType"
                    minOccurs="0" maxOccurs="unbounded"/>
      </xs:sequence>
      <xs:attribute name="isbn" type="xs:string" use="required"/>
    </xs:complexType>
  </xs:element>

  <xs:complexType name="CharacterType">
    <xs:sequence>
      <xs:element name="name" type="xs:string"/>
      <xs:element name="friend-of" type="xs:string" minOccurs="0"/>
      <xs:element name="since" type="xs:date"/>
      <xs:element name="qualification" type="xs:string"/>
    </xs:sequence>
  </xs:complexType>

</xs:schema>
```

---

4. XPath

Синтаксис за навигация в XML дърво.

Основни изрази

Израз	Значение
`/`	Корен на документа
`//`	Навсякъде в дървото
`.`	Текущ елемент
`..`	Родителски елемент

`*`	Всички елементи
`@attr`	Атрибут
`text()`	Текстово съдържание

Примери

```
/library/book                          <!-- всички book деца на library -->
//book                                 <!-- всички book навсякъде -->
/library/book[1]                       <!-- първата book -->
/library/book[@genre='fiction']        <!-- book с атрибут genre="fiction" -->
/library/book[@id='my_book']           <!-- book с id="my_book" -->
/library/book/title/text()             <!-- текстът на title -->
//book[price > 50]                     <!-- книги с цена > 50 -->
//book[last()]                         <!-- последната книга -->
//book[position() <= 3]                <!-- първите 3 книги -->
```

---

5. XSLT (XSL Transformations)

Трансформира XML в HTML, друг XML или текст.

Основна структура

```
<?xml version="1.0" encoding="UTF-8"?>
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:output method="xml" encoding="UTF-8" indent="yes"/>

  <xsl:template match="/">
    <!-- трансформация тук -->
  </xsl:template>

</xsl:stylesheet>
```

Ключови XSLT елементи

Елемент	Употреба
`<xsl:template match="...">`	Шаблон за съответстващи възли
`<xsl:apply-templates select="...">`	Прилага шаблони върху избрани деца
`<xsl:for-each select="...">`	Итерация по избрани елементи
`<xsl:value-of select="...">`	Извежда стойност (текст)
`<xsl:copy-of select="...">`	Копира елемент с деца

`<xsl:copy>`	Копира текущия елемент (без деца)
`<xsl:if test="...">`	Условие
`<xsl:choose>` + `<xsl:when>` + `<xsl:otherwise>`	if/else if/else
`<xsl:element name="...">`	Създава нов елемент динамично
`<xsl:attribute name="...">`	Добавя атрибут
`<xsl:variable name="...">`	Дефинира променлива
`<xsl:sort select="...">`	Сортиране

Пример: XSLT трансформация (изпит 2022)

Задача: ITEM с PRICE > 100 → копира се; ITEM с PRICE ≤ 100 → преобразува се в ITEM_CONVERTED (атрибути стават деца, деца стават атрибути).

```
<xsl:stylesheet version="1.0"
  xmlns:xsl="http://www.w3.org/1999/XSL/Transform">

  <xsl:output method="xml" encoding="UTF-8" indent="yes"/>

  <xsl:template match="/">
    <INVENTORY2>
      <xsl:for-each select="INVENTORY/ITEM">
        <xsl:choose>

          <!-- Условие: PRICE > 100 → копира се без промяна -->
          <xsl:when test="@PRICE &gt; 100">
            <xsl:copy-of select="."/>
          </xsl:when>

          <!-- Иначе: трансформиране в ITEM_CONVERTED -->
          <xsl:otherwise>
            <ITEM_CONVERTED>
              <!-- Под-елементите на ITEM → атрибути на ITEM_CONVERTED -->
              <xsl:for-each select="*">
                <xsl:attribute name="{name()}">
                  <xsl:value-of select="text()"/>
                </xsl:attribute>
              </xsl:for-each>
              <!-- Атрибутите на ITEM → под-елементи на ITEM_CONVERTED -->
              <xsl:for-each select="@*">
                <xsl:element name="{name()}">
                  <xsl:value-of select="."/>
                </xsl:element>
              </xsl:for-each>
            </ITEM_CONVERTED>
          </xsl:otherwise>

        </xsl:choose>
```

```
      </xsl:for-each>
    </INVENTORY2>
  </xsl:template>

</xsl:stylesheet>
```

Сравнение на оператори в XPath (в XSLT test=)

Значение	XPath в XSLT
>	>
<	<
>=	>=
<=	<=
=	=
!=	!=

Динамични имена с `{}`

```
<!-- {name()} → вмъква текущото име на елемент/атрибут -->
<xsl:attribute name="{name()}">...</xsl:attribute>
<xsl:element name="{name()}">...</xsl:element>
```

---

6. DOM vs SAX

	DOM	SAX
Модел	Зарежда цялото дърво в памет	Базиран на събития (event-driven)
Памет	Повече	По-малко
Подходящ за	Навигация, промени в документа	Четене на големи файлове
Посока	Двупосочна навигация	Само напред (forward-only)

Основни DOM интерфейси: `Node`, `Document`, `Element`, `Attr`, `Text`, `NodeList`

Основни SAX интерфейси: `ContentHandler`, `ErrorHandler`, `DTDHandler`, `EntityResolver`

---

7. Стратегия за решаване на задачи

Тип задача: „Напиши DTD по даден XML"

1. Намери **корена** → `<!ELEMENT root (...)>`
2. За всеки елемент: има ли деца? текст? атрибути?
3. Определи **реда** (sequence = запетая) и **броя** (`?` , `*` , `+`)

4. Атрибути: задължителен? → `#REQUIRED` ; незадължителен? → `#IMPLIED` ; ID/IDREF за референции
5. Изображения → `EMPTY` + `ENTITY` атрибут + `<!NOTATION>`
6. Напиши валиден XML документ, ако се иска

Тип задача: „Напиши XML Schema по даден XML"

1. Корен → `<xs:element name="root">` + `<xs:complexType>`
2. Деца в ред → `<xs:sequence>` ; произволен ред → `<xs:all>` ; избор → `<xs:choice>`
3. `minOccurs` / `maxOccurs` за брой
4. Атрибути: `use="required"` или `use="optional"`
5. Ако трябват ограничения → `<xs:simpleType>` + `<xs:restriction>` + фасети

Тип задача: „Напиши XSLT трансформация"

1. Започни с `<xsl:stylesheet>` + `<xsl:output>`
2. `<xsl:template match="/">` за корена
3. Итерирай с `<xsl:for-each select="...">` или `<xsl:apply-templates>`
4. За условия → `<xsl:choose>` / `<xsl:if>`
5. Копирай без промяна → `<xsl:copy-of select="."/>`
6. Динамично създавай елементи/атрибути → `<xsl:element name="{name()}">` / `<xsl:attribute name="{name()}">`
7. Помни: `>` в `test=` се пише като `>`

---

8. Чести грешки

- Забравен затварящ таг → XML невалиден
- `<!ELEMENT` с `(#PCDATA | child)*` — `#PCDATA` **задължително** е първи при смесено съдържание
- В DTD: `IDREF` реферира `ID` на **друг елемент** (не текст)
- В XSLT: `@*` = всички атрибути, `*` = всички под-елементи
- `<xsl:copy-of>` копира **целия поддървесен клон**; `<xsl:copy>` — само текущия елемент
- XML Schema: `maxOccurs="unbounded"` (не `"*"`)